
CS1004 - Object Oriented Programming

LAB # 2

POINTER

Lab Objectives:

- Understand pointer fundamentals and memory addresses
- Master the concepts of aliasing and referencing
- Practice pass by value vs pass by reference
- Implement pointer arithmetic and array manipulation
- Develop problem-solving skills using pointers

CONCEPT REVIEW:

1. Address Operator (&)

The & operator is used to get the memory address of a variable. Every variable in your program is stored at a specific location in memory, and the address operator allows you to access that location.

Example:

```
int x = 10;  
cout << &x; // Prints memory address like: 0x7ffd5c8e4a4c
```

2. Pointer (*)

A pointer is a special variable that stores the memory address of another variable. Instead of holding a direct value like 10 or 'A', it holds the location where that value is stored.

Declaration:

```
int *ptr;    // Declares a pointer to an integer  
double *dptr; // Declares a pointer to a double  
char *cptr;  // Declares a pointer to a character
```

Initialization:

```
int x = 10;  
int *ptr = &x; // ptr now holds the address of x
```

3. Dereference/Redirect Operator (*)

The * operator, when used with an existing pointer, accesses or modifies the value at the address stored in the pointer. This is called dereferencing.

Example:

```
int x = 10;
int *ptr = &x;
cout << *ptr; // Prints 10 (value at the address)
*ptr = 20;    // Changes x to 20
```

4. Reference/Alias (&)

A reference is an alias (another name) for an existing variable. Once initialized, it always refers to the same variable and cannot be changed to refer to another variable.

Example:

```
int x = 10;
int &ref = x; // ref is now another name for x
ref = 20;    // x is now 20
cout << x;   // Prints 20
```

Key Differences between Pointers and References:

- Pointers can be reassigned; references cannot
- Pointers can be NULL; references must always refer to a valid object
- Pointers need explicit dereferencing (*); references are automatically dereferenced

5. Pass by Value

When you pass by value, a copy of the variable is passed to the function. Any changes made inside the function do not affect the original variable.

Example:

```
void increment(int x) {
    x = x + 1; // Only the copy is modified
}
int main() {
    int num = 5;
    increment(num);
    cout << num; // Still prints 5
}
```

6. Pass by Reference

When you pass by reference (using & or pointers), the function receives access to the original variable. Changes made inside the function affect the original variable.

Using References:

```
void increment(int &x) {
    x = x + 1; // Modifies the original
}
```

```
int main() {  
    int num = 5;  
    increment(num);  
    cout << num; // Prints 6  
}
```

Using Pointers:

```
cpp  
void increment(int *x) {  
    *x = *x + 1; // Modifies the original  
}  
int main() {  
    int num = 5;  
    increment(&num);  
    cout << num; // Prints 6  
}
```

LAB TASKS

Task 1: Understanding Memory Addresses

Write a program that:

- Declares five integer variables with values 10, 20, 30, 40, 50
- Displays each variable's value and its memory address
- Calculates and displays the byte difference between consecutive addresses

Sample Output:

Variable 1: Value = 10, Address = 0x7ffd5c8e4a40

Variable 2: Value = 20, Address = 0x7ffd5c8e4a44, Offset = 4 bytes

Variable 3: Value = 30, Address = 0x7ffd5c8e4a48, Offset = 4 bytes

Variable 4: Value = 40, Address = 0x7ffd5c8e4a4c, Offset = 4 bytes

Variable 5: Value = 50, Address = 0x7ffd5c8e4a50, Offset = 4 bytes

Task 2: Multiply by Factor

Write a function multiplyByFactor() that takes an integer pointer and a factor (by value) as parameters, and multiplies the value by the factor. In main, take a number and factor from user, display before and after.

Sample Input:

Enter a number: 15

Enter factor: 4

Sample Output:

Before: 15

After: 60

Task 3: Area and Perimeter Calculator

Write a function `calculateRectangle()` that takes length and width as parameters (pass by value), and two reference parameters for area and perimeter. Calculate and store area and perimeter in the reference variables. In main, take length and width from user and display results.

Sample Input:

Enter length: 12

Enter width: 8

Sample Output:

Area: 96

Perimeter: 40

Task 4: Reference as Another Name

Write a program that:

- Creates an integer variable `score` with value 85
- Creates a reference `marks` to `score`
- Displays both with their addresses
- Adds 15 to `marks`
- Displays both again to show they share the same value and address

Sample Output:

Initial:

`score = 85, Address = 0x7ffd5c8e4a4c`

`marks = 85, Address = 0x7ffd5c8e4a4c`

After adding 15 to `marks`:

`score = 100, Address = 0x7ffd5c8e4a4c`

`marks = 100, Address = 0x7ffd5c8e4a4c`

Task 5: Pointer Redirection

Write a program that:

- Creates three float variables: `x=1.1, y=2.2, z=3.3`
- Creates a pointer and points it to `x`, displays value
- Redirects pointer to `y`, displays value
- Redirects pointer to `z`, displays value
- Uses pointer to modify all three variables

Sample Output:

Pointer to `x`: 1.1

Pointer to `y`: 2.2

Pointer to `z`: 3.3

After modifications through pointer:

`x = 5.5, y = 6.6, z = 7.7`

Task 6: Find Smaller Number

Write a function `findSmaller()` that takes two integers by reference and returns a reference to the smaller one. In main, take two numbers, get reference to smaller, add 50 to it, display both numbers.

Sample Input:

Enter two numbers: 85 42

Sample Output:

Original: num1 = 85, num2 = 42

After adding 50 to smaller:

num1 = 85, num2 = 92

Task 7: Square and Cube Calculator

Write a function `calculatePowers()` that takes a number (by value) and two pointers for square and cube. Calculate and store both values. In main, take a number and display its square and cube.

Sample Input:

Enter a number: 7

Sample Output:

Number: 7

Square: 49

Cube: 343

Task 8: Cyclic Swap

Write a function `cyclicSwap()` that takes four integer pointers and performs cyclic rotation ($a \rightarrow b$, $b \rightarrow c$, $c \rightarrow d$, $d \rightarrow a$). In main, create four variables, display them, perform cyclic swap twice, display after each swap.

Sample Input:

Enter four numbers: 5 10 15 20

Sample Output:

Original: a=5, b=10, c=15, d=20

After 1st swap: a=20, b=5, c=10, d=15

After 2nd swap: a=15, b=20, c=5, d=10

Task 9: Weight Conversion System

Write the following functions using pointers and references:

- `kgToPounds()` - takes kg by reference, pounds pointer, converts (1 kg = 2.205 pounds)
- `kgToGrams()` - takes kg pointer, grams pointer, converts
- `poundsToOunces()` - takes pounds by reference, ounces pointer, converts (1 pound = 16 ounces)
- `convertAllWeights()` - takes weight in kg and four pointers for kg, grams, pounds, ounces

In main, take weight in kg, perform all conversions.

Sample Input:

Enter weight in kg: 5

Sample Output:

Weight in kg: 5
Weight in grams: 5000
Weight in pounds: 11.025
Weight in ounces: 176.4
Converting 10 kg to all units:
Kilograms: 10
Grams: 10000
Pounds: 22.05
Ounces: 352.8

Task 10: Profit and Loss Calculator

Create a program with the following functions using pointers and references:

- `calculateProfit()` - takes cost price and selling price by reference, profit pointer
- `calculateLoss()` - takes cost price and selling price pointers, loss pointer
- `calculateProfitPercentage()` - takes profit and cost price by reference, percentage pointer
- `breakEvenAnalysis()` - takes cost, selling price, units sold as pointers, calculates total profit/loss
- `compareProducts()` - takes two cost prices and two selling prices by reference, compares profitability

In main, create multiple product scenarios and analyze profitability.

Sample Input:

Product 1:
Cost Price: 800
Selling Price: 1000
Units Sold: 50
Product 2:
Cost Price: 1200
Selling Price: 1500
Units Sold: 30
Product 3:
Cost Price: 500
Selling Price: 450
Units Sold: 40

Sample Output:

Product 1 Analysis:
Cost Price: 800
Selling Price: 1000
Profit per unit: 200
Profit percentage: 25%
Units sold: 50
Total profit: 10000
Product 2 Analysis:
Cost Price: 1200
Selling Price: 1500
Profit per unit: 300
Profit percentage: 25%



Units sold: 30

Total profit: 9000

Product 3 Analysis:

Cost Price: 500

Selling Price: 450

Loss per unit: 50

Loss percentage: 10%

Units sold: 40

Total loss: -2000

Comparison:

Product 1 total profit: 10000

Product 2 total profit: 9000

Product 3 total loss: -2000

Best performing: Product 1

Worst performing: Product 3

Overall profit/loss: 17000